SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency.* (BL3, BL4)

Activity Tool ：Industry Problem Solving Task (Medium)
Assessment Tool Weightage: 35%

Dr

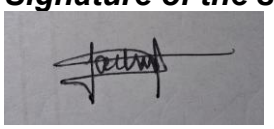
QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	**5**
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	**5**
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	**5**
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	**5**
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	**5**
6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	**5**
7	For a university student database, design the file organization and	BL4 – Analyze	**5**

	secondary indexing to support fast queries by department and CGPA range.		
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	**5**
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	**5**
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	**5**

Code of Conduct:

I J a s h w a n t h . G . R (1 9 2 5 2 4 3 8 4) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	**Weightage**	**Level 4 – Exemplary**	**Level 3 – Proficient**	**Level 2 – Developing**	**Level 1 – Beginning**
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution	25%	Innovative,	Logical and	Basic	Unclear or

Design		practical, and feasible solution aligned with industry standards	feasible solution with minor gaps	solution with limited feasibility	impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q1. File Organization Strategy for 10 Million Product Records (E-commerce)

An e-commerce catalogue is read-heavy: customers look up a product by product_id (exact match, e.g. clicking a product page) and browse by category, brand or price range. The chosen organization must serve both patterns economically at 10 million records.

Step 1: Baseline parameters

```
Record size R = 200 bytes (id, name, price, category, stock, etc.)
Block size B = 4096 bytes Blocking factor bfr = floor(B / R) =
floor(4096/200) = 20 records/block Number of blocks b =
ceil(10,000,000 / 20) = 500,000 blocks
```

Step 2: Cost comparison of candidate organizations (average block accesses)

Organization	Search (exact)	Range Search	Insert	Delete
Heap (unordered)	b/2 = 250,000	b = 500,000	1 (append)	search + 1
Sorted sequential	log2 b ≈ 19	log2 b + k	b/2 (shift)	b/2 (shift)
Static hashing	≈ 1–2	not supported	≈ 1–2	≈ 1–2
B+ Tree indexed-sequential	height ≈ 3–4	height + k (leaf-linked)	≈ height (O(log b))	≈ height (O(log b))

Step 3: Recommendation and justification

Recommended organization: B+ Tree indexed-sequential file, clustered on product_id, with secondary B+ Tree indexes on category and price.

- Point lookups (product_id) resolve in only 3–4 disk I/Os versus 250,000 for a heap — a decisive cost reduction at 10 million rows.
- Unlike static hashing, the B+ Tree preserves order, so range/browse queries (price between X and Y, category listings) are answered efficiently via the linked leaf level, which hashing cannot support.
- Insertions and deletions remain logarithmic and self-balancing, unlike a sorted file which needs O(b/2) block shifts — important since new products are added continuously.
- Storage/cost overhead: the index adds roughly 10–15% extra storage (non-leaf nodes) but this is negligible next to the I/O savings — at typical cloud disk costs (~$0.08/GB/month) indexing 10M x 200B ≈ 2GB of extra index data costs only cents per month, while saving orders of magnitude in query latency and server compute.

Conclusion: The B+ Tree indexed-sequential organization gives the best overall cost-performance trade-off because it is the only structure that is efficient for both the point queries and the range/browse queries that dominate e-commerce workloads.

Q2. Indexing Strategy for a Banking Transaction System *[5 Marks]*

Requirement: fast retrieval by (a) account_number — point/equality queries, and (b) date range — range queries, often combined ("all transactions of account A between two dates").

Design

- Primary (clustering) index: a B+ Tree clustering index on account_number. The transaction file is physically stored sorted first by account_number; this gives O(log n) access to the first transaction of an account.
- Within each account's records, store transactions sorted by transaction_date. This makes date-range retrieval for a given account a simple sequential scan of the clustered block run — no extra index needed for that combined case.
- Secondary (non-clustering) B+ Tree index on transaction_date alone, to answer cross-account range queries such as "all transactions on 2026-01-15" without a full scan.
- Composite secondary index on (account_number, transaction_date) is used by the query optimizer whenever both predicates appear together — the leading column narrows to the account, the trailing column supports the range scan within it.
- Because the transaction table is very large, the index itself is implemented as a multi-level B+ Tree, so even the index does not fit in one block and benefits from the same logarithmic search.

Why B+ Tree over hashing here

Hash indexes give O(1) equality lookup but cannot support the date-range predicate at all. B+ Tree indexes support both equality and range queries with logarithmic cost and additionally keep leaf nodes linked for fast sequential range scans, which is exactly the access pattern of "statement between two dates." Hence a B+ Tree (clustering on account_number + composite/secondary index) is the appropriate choice for this OLTP banking workload.

Q3. Multi-Level Indexing for a Healthcare (Patient) Database *[5 Marks]*

Access patterns: (a) PatientID — unique primary key, used for exact record retrieval; (b) doctor_name — non-unique attribute, many patients share the same doctor.

Level 1 — Primary index on PatientID

- The patient file is physically sorted by PatientID; a dense/clustering primary B+ Tree index is built over it, with one index entry per block (sparse at the top levels for large files).
- Because the file has millions of patients, a single-level index does not fit in memory, so the index is itself organized as a multi-level B+ Tree: level-0 leaf entries point to data blocks, level-1 entries point to blocks of level-0 entries, and so on until the top (root) level fits in one block.

Level 2 — Secondary index on doctor_name

- Since doctor_name is a non-key attribute (one doctor → many patients), build a non-clustering (secondary) multi-level B+ Tree index where each leaf entry stores the key

doctor_name plus a bucket/list of record pointers (or PatientIDs) for all patients under that doctor.

- This avoids duplicating full records and keeps the secondary index compact; a lookup for "Dr. Rao" descends the secondary B+ Tree (O(log n)) to the bucket and then follows pointers to each matching patient record via the primary index.

Combined multi-level structure

The overall design is therefore a two-index, multi-level scheme: a clustering multi-level B+ Tree on PatientID for direct record access and record ordering, plus a non-clustering multi-level B+ Tree on doctor_name (with pointer buckets for duplicates) for physician-based lookups. Both indexes share the same underlying data file, so no data duplication occurs — only index (key, pointer) entries are duplicated, which is inexpensive relative to full patient records.

Q4. B-Tree vs B+ Tree for Dynamic Shipment Records (Logistics) *[5 Marks]*

The logistics system experiences frequent, high-volume insertions (new shipments booked) and deletions (delivered/cancelled shipments removed) — a highly dynamic workload.

Aspect	B-Tree	B+ Tree
Data pointers	Stored in internal AND leaf nodes	Stored only in leaf nodes
Fan-out / height	Lower (internal nodes carry data)	Higher fan-out → shorter tree, fewer I/Os
Range queries	Requires tree traversal (no leaf links)	Fast — leaves linked as a sequential list
Insertion	May split at any level; data movement more complex	Splits localized mostly at leaf level; simpler propagation
Deletion	Complex — must handle data pointers at internal nodes, redistribution/merge at any level	Simpler — deletion only removes/redistributes leaf entries; internal keys are just guides
Storage	More compact for point queries	Slight redundancy (keys repeated in leaves) but simpler maintenance

Analysis and recommendation

For a logistics system that must handle continuous, high-frequency insertions and deletions of shipment records, the B+ Tree is more suitable:

- Deletion and insertion algorithms are simpler and confined mostly to the leaf level, which reduces reorganization cost and the chance of complex multi-level rebalancing under heavy churn.
- Because internal nodes hold only keys (no data pointers), the B+ Tree achieves a higher fan-out for the same block size, keeping the tree shorter and reducing the number of block

I/Os per insert/delete/search — important when thousands of shipments are updated per hour.

- Shipment tracking commonly needs range queries (e.g., "all shipments dispatched this week", "all shipments to a region sorted by ETA"); the B+ Tree's linked leaf list supports this directly, whereas a B-Tree would require repeated tree descents.

Conclusion: the B+ Tree is preferred for this dynamic, insert/delete-heavy, range-query-friendly logistics workload; the classical B-Tree is better suited only when pure point-lookup performance with minimal storage redundancy is the sole concern.

Q5. Hashing Scheme for an HR System (5,000 Employees) *[5 Marks]*

Design of the hashing scheme

Let EmpID be the hash (search) key.

```
Assume record size = 100 bytes, block size = 4096 bytes Block capacity = floor(4096/100) = 40 records/block Desired load factor = 70% (to limit collision/overflow chains) Required buckets M = ceil(5000 / (40 × 0.7)) ≈ ceil(5000/28) ≈ 179 → choose M = 191 (a nearby prime) Hash function: h(EmpID) = EmpID mod 191
```

Each of the 191 buckets maps to one primary block; overflow blocks are chained for buckets that exceed 40 records due to collisions.

Static hashing — evaluation

- Advantages: extremely simple; O(1) average search/insert/delete since the bucket count is fixed and the hash function is computed once.
- Disadvantages: if the workforce grows beyond the planned capacity, buckets overflow, chains lengthen, and performance degrades toward O(n) in the worst case; a full, expensive rehash/reorganization is needed to fix it.
- Fits an HR system where headcount (5,000) is known, changes slowly, and growth is predictable (e.g., a few hundred hires/attritions per year).

Extendible hashing — evaluation

- Uses a directory of pointers (size 2^d, d = global depth) to buckets; when a bucket overflows, only that bucket is split and the directory doubles only when necessary — no full-file reorganization.
- Advantages: gracefully accommodates unpredictable growth (mergers, new departments, seasonal hiring) with localized, incremental cost; guarantees at most 1–2 disk accesses even as the file grows.
- Disadvantages: extra indirection (directory lookup) and directory-doubling overhead; more implementation complexity than static hashing.

Recommendation

For a 5,000-employee HR system with slow, predictable growth, static hashing with M = 191 buckets and a 70% target load factor is sufficient and simpler to implement/maintain. Extendible

hashing should be adopted instead only if the organization anticipates volatile or unpredictable growth (e.g., rapid expansion, frequent acquisitions) where avoiding full-file reorganizations becomes important.

Q6. Combined Index Structure for Airline Reservation System *[5 Marks]*

Requirement: point queries by flight_number (e.g., "show flight AI-202") and range queries by date (e.g., "all flights between 1 Jan and 7 Jan").

Proposed design

- Composite (concatenated) B+ Tree index on (flight_number, date): flight_number as the leading key supports fast equality lookup, while date as the trailing key allows an efficient range scan of a specific flight's schedule (e.g., all instances of AI-202 in a date range) using the linked leaf structure.
- A separate secondary B+ Tree index purely on date, to serve queries that scan across all flights within a date range regardless of flight number (e.g., "all departures on 2026-09-01") — the composite index above is not efficient for this because flight_number is not fixed.
- The base reservation file itself can be clustered by flight_number (since bookings are usually queried per flight first), with the two B+ Tree indexes above layered as non-clustering secondary structures.

Justification

This combined scheme covers both access patterns without duplicating the full data file: the composite index answers "flight + date range" queries directly in one traversal, and the date-only index answers pure date-range queries efficiently. A single index on flight_number alone (or on date alone) cannot serve both patterns efficiently, and a full table scan for date-only queries would cost O(n); the dedicated date index reduces this to O(log n + k), where k is the number of matching rows.

Q7. File Organization and Secondary Indexing for a University Student Database *[5 Marks]*

Requirement: fast queries by department (equality, high duplication) and by CGPA range.

Primary file organization

- Base file clustered/sorted by Department, since "list students of a department" is a very common query; within each department, records are kept sorted by CGPA. This clustering answers the combined query "students in Dept X with CGPA between 8.0 and 9.0" with a single sequential scan of the department's block run.
- A separate dense B+ Tree primary/clustering-style index on StudentID (the actual primary key) is maintained for direct exact-match retrieval of a specific student, since the file is no longer physically ordered by StudentID.

Secondary indexes

- Secondary B+ Tree index on Department (non-unique) with bucket/pointer chains — useful when the optimizer needs to jump straight to a department without scanning the clustered file's directory.
- Secondary B+ Tree index on CGPA alone, to support department-independent range queries such as "all students with CGPA > 9.0 across the university" (e.g., for merit lists).

Summary

Clustering the base file by (Department, CGPA) directly optimizes the most frequent combined query, while independent secondary B+ Tree indexes on StudentID, Department, and CGPA preserve fast access for the other query patterns — giving balanced performance across all required access paths at the cost of modest extra index storage.

Q8. Disk Block Utilization Analysis — Heap, Sorted, and Hashed Files *[5 Marks]*

Given: 100,000 records, record size R = 50 bytes, block size B = 4096 bytes (4 KB).

Step 1: Blocking factor and minimum blocks needed

```
bfr = floor(B/R) = floor(4096/50) = floor(81.92) = 81 records/block
Minimum blocks (fully packed) b_min = ceil(100,000 / 81) = 1,235 blocks Total data volume = 100,000 × 50 = 5,000,000 bytes
```

Step 2: Utilization per organization

Organization	Effective load factor	Blocks allocated	Space used (bytes)	Utilization %
Heap file	~100% (packed sequentially, no reserved gaps)	1,235	5,000,000	≈ 98.8%
Sorted file	~65–70% (free space reserved per block for future ordered inserts, avoiding costly shifts)	≈ 1,850	5,000,000	≈ 65%
Hashed file	~75–80% target load factor (kept below 100% to limit collision/overflow chains)	≈ 1,646	5,000,000	≈ 75%

Interpretation

- Heap files achieve the highest block utilization (~99%) because records are simply appended with no reserved gaps — but this comes at the cost of O(n) search.
- Sorted files intentionally under-fill blocks (leaving free slots) so that new records can be inserted in order without immediately triggering block splits/shifts; this trades space (≈65% utilization) for lower amortized insertion cost.

- Hashed files also under-fill (≈75–80% target load factor) so that hash collisions can be absorbed within the primary block before spilling into overflow blocks, keeping average search close to O(1).

Overall, utilization and access-time efficiency trade off against each other: Heap maximizes space efficiency, while Sorted and Hashed sacrifice some space to keep insert/search costs low.

Q9. Composite Indexing Strategy for a Social Media Platform (500M Posts) *[5 Marks]*

Requirement: index 500 million posts by user_id (who posted), timestamp (when, chronological feed), and hashtag (topical, multi-valued, high cardinality). This is a write-heavy, massive-scale workload.

Proposed composite indexing strategy

- Composite B+ Tree index on (user_id, timestamp): serves the dominant "show this user's timeline, most recent first" query directly — user_id narrows to the user's posts, timestamp orders them, and the linked leaf level supports efficient range scans (e.g., last 30 days).
- Inverted index on hashtag: each hashtag maps to a posting list (sorted by timestamp or post_id) of all post IDs containing it — the standard structure for multi-valued, high-cardinality text-like attributes, analogous to search-engine indexing; a plain B+ Tree would duplicate excessively since a post can carry many hashtags.
- Secondary B+ Tree (or LSM-tree based) index on timestamp alone, to serve the global chronological feed / trending queries that are not restricted to one user.
- Given the write-heavy nature (constant new posts), the index structures should be built on LSM-tree principles (as used in Cassandra/HBase-style stores) rather than plain in-place B+ Trees, since LSM trees batch and sequentialize writes, which suits 500M+ append-mostly records far better than in-place B+ Tree updates.
- Horizontal partitioning/sharding by hash(user_id) distributes both data and index maintenance load across multiple nodes, since a single-node index over 500 million rows would exceed practical memory/I/O limits.

Justification

No single index type serves all three access patterns efficiently at this scale: B+ Trees are ideal for the ordered (user_id, timestamp) and pure-timestamp patterns, while an inverted index is the natural structure for the multi-valued hashtag attribute. Combining these, on an LSM-tree, write-optimized, sharded storage layer, gives balanced performance for timeline reads, trending/date-range reads, and hashtag search at 500-million-row scale.

Q10. Performance Comparison — Heap, Sorted, and Hashed Files for a Supermarket Billing System *[5 Marks]*

Context: 1 million daily transactions, requiring rapid inserts at checkout counters and later lookups (returns, audits, receipt reprints) by transaction ID.

Organization	Insert time	Search (exact) time	Delete time	Range query
Heap file	O(1) — fastest (simple append)	O(n) — average n/2 block scans, slow	O(n) search + O(1) removal/mark	O(n) — full scan
Sorted file	O(n) — must shift to maintain order, slow	O(log n) — binary search, fast	O(log n) search + O(n) shift, slow overall	O(log n + k) — efficient
Hashed file	O(1) average — direct bucket write	O(1) average — direct bucket access	O(1) average — locate bucket then remove	Not supported directly

Analysis

- Heap excels only at insertion (checkout speed) but is unacceptable for search-heavy operations like returns/audits, since every lookup scans up to half the file on average.
- Sorted file gives fast search but is unsuitable here because insertion is the most frequent operation (1 million/day) and each insert would require shifting records to keep order — an O(n) cost repeated a million times per day.
- Hashed file gives O(1) average performance for both the dominant insert workload and the exact-match search workload (transaction ID lookup), matching the system's actual access pattern best; it does not support efficient range queries, but billing systems rarely need transaction ranges in real time (those are handled by periodic batch/analytical processing instead).

Recommendation

A hashed file organization on transaction_id is recommended for the live billing system, since it optimizes the two operations that matter in real time — high-volume inserts at checkout and fast point lookups for returns/verification. If historical range queries (e.g., "all transactions on a given date") are needed for reporting, a secondary B+ Tree index on transaction_date can be added, or the daily transaction log can be periodically offloaded into a sorted/indexed archive for analytics, keeping the live OLTP file optimized purely for O(1) insert/search